03 — Post-Training: Data, LoRA/QLoRA, Hardware, and Validation

This is the deep post-training plan for the **4×A100 40 GB** remote box plus Kaggle GPUs. It is deliberately conservative: the public evidence includes a prompt-only 31B notebook that *claims* **0.24 public** (S36; author-reported), while a 1,500-trajectory LoRA experiment did **not** beat the base model (S25). Post-training is an optimization to attempt after the baseline works, not a shortcut around a weak scaffold.

The goal is not "train a model." The goal is: **produce an adapter that improves resolved tasks per unit of global time on the hidden private-repo distribution, without breaking startup, context, or the 12-hour limit.**

---

1. Decision summary

Decision	Recommendation	Why
First artifact	Prompt-only 31B bundle	A public notebook claims 0.24; no training risk (S36)
Training method	LoRA or QLoRA SFT on successful trajectories	Cheapest path to behavior change; supported by competition (S1, S5:191-201)
Base for training	Unquantized QAT or BF16 Gemma 4 31B	PEFT cannot attach to the packaged W4A16 CT checkpoint (S25, S45)
Serving base	`gemma-4-31b-it-qat-w4a16-ct`	Fixed by competition (S1)
Adapter rank	16 primary; 8 only with alpha 4/8; 32 only with evidence	W4A16 rank-32 instability report; rank-64 host-seen but not proof of effectiveness (S24, S26)
Alpha	Rank 16, alpha 16 (scale 1.0) start; sweep 8/16 (scale 0.5/1.0); 32 diagnostic	Public case: scale 2.0 degenerate, scale 1.0 normal, 0.5 safe (S25)
Target modules	q/k/v/o/gate/up/down, text backbone only	Standard LoRA coverage; vLLM supports these (S12, S22)
Sequence length	8k–16k per-call samples, set from measured p95	[ESTIMATE] 32k is expensive; competition context is 32k but training at 32k is a stretch on 4×40 GB (S11/S13 show 32k on multi-GPU; S25 used 32k)
Data size	300–500 resolved trajectories; 2,000–6,000 per-call samples	Public 697-sample run did not beat base; quality/scale both matter (S25, S18–S20)
Thinking	Match training and inference	Public mismatch caused huge thoughts/timeouts (S25, S8)

Decision	Recommendation	Why
Success criterion	≥8 discordant pairs and ≥75% wins (or two seeds) + better p95/patch rate, then a hidden score	Local-to-hidden transfer is weak; one score is noise (S34, S35, S39)

If you cannot satisfy the data/validation prerequisites, do not train. Spend the time on prompt/budget improvements and the paper track instead.

---

2. What “post-training” means in this competition

The allowed submission can contain (S5:63-82, S5:191-201):

- a declarative agent config;
- optional PEFT LoRA adapters (`adapter_config.json` + `adapter_model.safetensors`);
- optional skills and sub-agents.

It cannot contain arbitrary training code or a custom inference server. The scorer loads the fixed base model and applies your adapter(s) via vLLM (S5:187-195). Therefore “post-training” here means:

1. **SFT/LoRA** on trajectories from a stronger or same-model agent.
2. **Preference/RL** methods only if you can build a reliable reward and afford the compute.
3. **Distillation** from stronger models into the allowed Gemma 4 base.

Full fine-tuning is not practical: 31B BF16 weights are ~62 GB, and full optimizer states are much larger (S7, S11). LoRA/QLoRA is the intended path.

---

3. The base-model problem (read this before training)

The competition base is `gemma-4-31b-it-qat-w4a16-ct`: a **compressed-tensors W4A16 inference checkpoint** for vLLM (S7, S43, S44). It is not a standard trainable HF checkpoint.

Public evidence:

- PEFT could not attach to the packaged `-qat-w4a16-ct` checkpoint; the experiment trained on a **dequantized BF16 copy** instead (S25).
- Unsloth’s QAT discussion says there is no supported way to QLoRA the actual QAT checkpoint while preserving its Q4_0 calibration; the safer path is to train on the unquantized QAT release and keep the adapter separate (S45).
- QLoRA uses NF4/int4, which is not Google’s Q4_0 quantizer; loading `-qat-q4_0-unquantized` with `load_in_4bit=True` requantizes for training (S45).

Practical rule:

> Train the adapter on a trainable BF16 checkpoint. The verified ID in Axolotl’s official Gemma 4 config is `google/gemma-4-31B` (S12). The official QAT unquantized checkpoint from the Gemma 4 QAT

> collection (`...-qat-q4_0-unquantized`; S7/S43) is the closest representation of the scored base, but verify its exact repo ID and revision before download. Serve the adapter on the competition alias `gemma-4-31b-it-qat-w4a16-ct` (the Hugging Face repo is `google/gemma-4-31B-it-qat-w4a16-ct`; casing differs). Pin the exact revisions. Validate that the adapter is active and effective on the CT base; do not assume the training-base behavior transfers.

Why the unquantized QAT checkpoint: it is the half-precision weights extracted from the QAT pipeline, intended for custom downstream compilation/research (S7, S43). It is the closest trainable representation of the scored base. If its exact repo ID/revision is not verified or your trainer does not accept it, use `google/gemma-4-31B` BF16 (S12) and record the difference.

Do **not** merge the adapter into the CT weights and re-quantize unless you have a verified pipeline and can still submit only the allowed base + adapter. The competition serves the base separately; merging changes the artifact and may violate the submission contract.

---

4. Data: what to train on

4.1 Source of trajectories

The best data is **your own 31B agent's successful trajectories on the public tasks**, generated with the same prompt, tool schemas, and thinking mode you will use at inference. This matches the deployment distribution and avoids licensing surprises.

Define the units first:

- **Task**: one public `instance_id` (129 available).
- **Trajectory**: one complete agent attempt on one task (many per task if you sample multiple temperatures).
- **Sample**: one assistant turn inside a trajectory (thought + text + tool call); roughly 7–15 per trajectory.

Process:

1. Take the 129 public tasks and snapshots (S2).
2. Run the reference agent (the public prompt-only config or your tuned baseline) at five temperatures between 0.2 and 1.0 (e.g. 0.2, 0.4, 0.6, 0.8, 1.0).
3. Save the harness artifacts: `patches/`, `test_outputs/`, `traces/`, `task_results.jsonl` (S5:608-620).
4. Keep only `resolved=True` trajectories.
5. For each task, keep **top-k (k=2–4) successful trajectories** by quality, not only the single shortest. With 129 tasks this yields roughly **300–500 trajectories**, which expand to roughly **2,000–6,000 per-call samples**.
6. Drop tasks/repos where the local verifier misbehaved (environment failures, broken snapshots).
7. Deduplicate near-identical trajectories.

The public 1,500-trajectory experiment's "697 examples from 99 tasks" are **samples**, not trajectories (S25). Do not confuse the units when sizing a run.

The public 1,500-trajectory experiment used five temperatures, kept resolved runs, kept the shortest per task (we recommend top-k), dropped broken repos, and ended with **697 per-call training samples from 99 tasks + 60 held-out samples from 11 tasks** (S25). That is a useful scale reference, but it did not beat the base. More data alone is not a solution.

4.2 Thinking-mode consistency

This is a common, expensive mistake.

- The public prompt-only config uses `thinking_budget: 4096`, `include_thoughts: true` (S36).
- The public LoRA experiment trained on trajectories with **no thinking traces** but ran inference with thinking 4096; at alpha 16 it produced enormous thought blocks and timeouts (S25).
- S10 uses the thinking template for 31B/26B and the non-thinking `gemma-4` template for the small models; the model card describes the thought-token format (S8).

Choose one mode and use it everywhere:

- **Recommended:** generate trajectories with thinking enabled, include the thought tokens in the training context/labels, and run inference with thinking enabled.
- **Alternative:** generate and serve with thinking disabled; you lose some tool-call reliability but avoid mismatch.
- Do not train without thoughts and infer with thoughts.

If you include thoughts in training, preserve the exact `<|channel>thought\n...<channel|>` tokens and the harness's history behavior. The model card says historical thoughts should not be added before the next user turn, except tool-call turns where thinking should be preserved (S8). The harness keeps thoughts in tool-call context when `include_thoughts: true` (S5:136). Match that.

Freeze the prompt and tool schema across the adapter line. Record `prompt_sha256` and `tool_schema_sha256`; use the same values for trajectory generation, adapter evaluation, and submission. If you change the prompt after generating data, either regenerate a 30-task calibration slice or drop the adapter; otherwise a gain/loss is uninterpretable.

4.3 What a training sample is

Do not train on "the final patch" alone. The agent's behavior is a sequence of model calls with tool observations. The useful supervision is **each assistant turn**: thought + text + tool call.

For each model call in a trajectory:

- **Prompt/context:** the exact messages the model saw: system prompt, user task, prior assistant turns, tool calls, tool observations.
- **Completion:** the assistant's output for this call: thought channel (if enabled), text, and tool-call tokens.
- **Labels:** `-100` for the prompt and tool observations; token IDs for the assistant completion.
- **Tool schemas:** the same nine tool definitions used at inference.
- **Chat template:** the exact Gemma 4 template and special tokens.

Do not concatenate raw assistant outputs into one long string. The runtime re-renders history for each call; a concatenation trains on context the model never saw (S39 describes this exact pitfall). Preserve the

served prompt and output token IDs per call if possible; otherwise re-render each call with the exact template and verify token counts against the trace.

The harness writes ATIF-compatible traces with steps, tool calls, observations, and metrics (S17, S5:617). Use those steps as the source of truth. If token IDs are not in the trace, reconstruct them carefully and validate.

4.4 Assistant-only loss

You want loss on the assistant's tokens, not on the system prompt, user task, or tool outputs.

Options:

1. **TRL** `assistant_only_loss=True` with a conversational dataset. It requires `{% generation %}` / `{% endgeneration %}` markers in the chat template (S16). The stock Gemma 4 template may not have them; the public experiment patched a copy, verified the token stream was identical to stock (12,602 tokens in both), and used it (S25). This is the cleanest approach if you use TRL.
2. **Prompt-completion pairs** per model call, with `completion_only_loss=True` (default for prompt-completion datasets) (S16). Tokenize the prompt and completion separately, concatenate, and verify the result equals the full render.
3. **Pre-tokenized** `input_ids` / `attention_mask` / `labels` with explicit `-100` masks. Most control, most code. The public experiment switched to this to avoid TRL API drift (S25).

Whatever you choose, inspect a decoded sample with `-100` replaced by a pad token. You should see only the assistant tokens. If you see the user prompt or tool output, your mask is wrong.

4.5 Filtering and quality

Success is necessary but not sufficient. Successful trajectories can contain redundant exploration, wrong turns, risky commands, or test-specific hacks (S19). The public evidence:

- Trajectory quality/quantity trade-off is scale-dependent; error-retry rate is a dominant quality signal (S20).
- SWE-Prime: training on a 10% selected subset outperformed the full resolved dataset on SWE-Bench Pro/Verified (S19).
- SRFT: masking harmful steps in unresolved trajectories improved over RFT (30.9% → 32.2% on SWE-bench Verified) (S18).

A practical quality score:

Dimension	Signal	Prefer
Outcome	Harness `resolved`	True
Efficiency	Tool calls, wall time	Fewer calls, under budget
Style	First source edit call	By call 12 (S38)
Hygiene	Test/config edits, scratch files	None
Loopiness	Identical-call streaks	None

Dimension	Signal	Prefer
Localization	Files touched vs reference patch	Overlap (coarse, not proof)
Leakage	Reads outside `/workspace`	None
Thinking	Thought tokens present	Match inference

Filtering rules:

- Drop trajectories with test/config edits or reads outside `/workspace` (public wheel copies can leak fixes; hidden private repos do not have them) (S31).
- Drop trajectories with long identical-call streaks; they teach looping.
- Drop trajectories that hit the per-task cap without a patch.
- Drop environment-failure tasks (collection errors, missing fixtures) rather than teaching the model to fight a broken harness.
- Keep some error-recovery examples if you use SRFT-style step masking; otherwise prefer clean successes.
- Balance repositories; don't let one repo dominate.

4.6 Data size and diversity

There is no official minimum. Public evidence suggests:

- 697 per-call samples / 99 tasks was not enough to beat the base in one experiment (S25).
- SWE-agent SFT papers use thousands of trajectories (S18–S20), but they have larger compute and different models.
- Quality filtering can make a 10% subset outperform the full set (S19).

Plan:

- Start with **300–500 resolved trajectories** (2–4 per task), which is roughly **2,000–6,000 per-call samples**.
- If generation is cheap, 1,000 trajectories is a reasonable upper bound for a first campaign; more data with the same flaws will amplify the flaws.
- Hold out entire tasks/repos for evaluation; never train on the holdout.
- If the first adapter does not beat the base, improve data quality/thinking consistency before scaling to 10k examples.

4.7 Leakage and publication rules

- Public tasks include `patch` and `test_patch` (S2). Do not train on the hidden tests; do not put `test_patch` in the model context. Use it only for local verification.
- Public repos may be in the model's pretraining data. Hidden private repos reduce that leakage but do not eliminate it.
- Public wheelhouse may contain released repo versions with fixes; stronger models sometimes read them outside `/workspace` (S31). Drop trajectories that do this; hidden repos won't have the same leak.

- External data must be publicly available/equally accessible or meet the Reasonableness Standard; winners must provide training code/details (S3). Publish your trajectory dataset, generation script, filters, and licenses if you use it. Do not publish private competition data.

---

5. LoRA/QLoRA mechanics

5.1 The scaling factor

LoRA adds:

$$h = Wx + \frac{\alpha}{r} BAx$$

The scale `alpha/r` is applied at inference and is stored in `adapter_config.json`. It is not baked into the weights. Public evidence from a 31B LoRA on this competition (S25):

rank	alpha	scale	Observed behavior
16	32	2.0	Degenerate: 257 s before first tool call, repeated `git log`, no edit, score 0
16	16	1.0	Loops gone; more deliberate; thought blocks 2.5k–14.7k chars; median 290 s/task
16	8	0.5	Safest/fastest; median 163 s; always produced a patch; only a quarter of the learned correction

This is one experiment, not a universal law. But it is a strong reason to:

- train with `alpha = rank` (scale 1.0) as the default;
- sweep `alpha` at inference (8/16; 32 diagnostic) before concluding the adapter/data is bad;
- treat a degenerate adapter as a scaling problem first, not a data problem.

Because alpha is in `adapter_config.json`, you can change it without retraining and without changing the weights. This is a cheap experiment.

5.2 Rank and target modules

- Target **all seven text projections**: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj` (S12, S22).
- Restrict to the text backbone; do not adapt vision/audio towers (S11, S12).
- For 31B, all seven across 60 layers is ~410 modules; rank 16 is ~122.4M trainable parameters and ~245 MB (S25).
- Rank 8–16 is the conservative start; if you use rank 8, sweep alpha 4/8. Rank 32 may be unstable on compressed-tensors W4A16 (S24). The host has seen rank-64 adapters score (S26), but "scored" does not prove the adapter was effective; validate.
- README calls 128 `max_lora_rank`, but the notebook computes `max(detected_ranks)` with 128 only as a fallback and does not clamp (S5:195,201; S6:284). Do not ship >128. Rank 128 is ~0.9–1.8 GB; it fits the 3 GiB limit but may hurt KV cache/startup and is harder to train.

- `lora_dropout`: 0–0.05. `bias: none`. `use_rslora: false` unless you have a reason. `task_type: CAUSAL_LM`.

5.3 Adapter size budget

The submission is <3 GiB unpacked, including adapters (S5:115). Size planning for 31B all-linear (S5:197-201):

Rank	Approx. size	How many fit
16	110–220 MB (planning); **245 MB measured**	~8 planning; ~13 measured
32	220–450 MB	6–8
64	450–900 MB	3–6
128	0.9–1.8 GB	1–3

These are planning ranges; actual size depends on target modules and dtype. A measured rank-16 all-seven 31B adapter was ~245 MB (122.4M parameters at bf16; S25), so the 110–220 MB row is optimistic for full coverage and the ~8 fit count is conservative. Check `du -sh` after export and use the measured number for the 3 GiB budget.

5.4 The W4A16 serving risk

The competition base is W4A16 compressed-tensors. An upstream vLLM issue on a different W4A16 model found (S24):

- rank-32 all-layer adapters produced weak, non-reproducible outputs;
- rank-16 was reproducible;
- rank-8 targeting only the last 8 layers worked correctly.

This is not Gemma-specific, and the competition's vLLM build may differ. But it means:

- do not assume a larger rank is better;
- test rank 8/16 and partial coverage if rank 16 all-layer is unstable;
- A/B outputs at temperature 0 and check reproducibility across server restarts;
- if the adapter is weak/non-reproducible, try lower rank/partial coverage before abandoning the data.

5.5 Adapter loading can be silent

vLLM has had Gemma 4 bugs where the adapter loads but is ignored (S23). The fix was a vLLM PR; the competition build may or may not include it. Validate:

1. Serve base without adapter; generate at temperature 0 for 3 prompts.
2. Serve with adapter; generate the same prompts at temperature 0.
3. Compare hashes/outputs. They must differ. If identical, the adapter is not active.
4. Check the served base revision matches the adapter's training base (S23).
5. Check `adapter_config.json` target modules match the served model's module names.

Do not trust “Loaded new LoRA adapter” logs.

---

6. Hardware plan: 4×A100 40 GB

6.1 Preflight

Run on the remote box:

```
nvidia-smi
free -h
df -h
python --version
pip --version
git --version
nvcc --version 2>/dev/null || true
```

You need:

- 4×A100 40 GB visible; NCCL works.
- Enough RAM to load/shard the model and tokenize data; 64 GB+ preferred, more if CPU offload.
- Enough disk: 31B BF16 ~62 GB; QAT CT ~17.5 GB; data 22.42 GB; checkpoints/adapters/results 50–150 GB. Plan 250–400 GB free.
- A recent PyTorch/CUDA stack compatible with your trainer; Gemma 4 needs a Transformers version that knows `gemma4`.
- Internet for initial downloads; training itself can be offline after caching.

6.2 Option A — 31B QLoRA on one A100 40 GB (recommended first)

Why: Unsloth states 31B QLoRA works at ~22 GB (S10); Axolotl’s 31B QLoRA example reserves ~25.2 GiB on 1×80 GB at 2k sequence (S11/S12). A 40 GB card has room at 4k–8k sequence with gradient checkpointing, but you must smoke-test.

Use the other three A100s for parallel experiments: different ranks, alphas, data filters, or seeds. This avoids multi-GPU FSDP/QLoRA compatibility issues with Gemma 4.

Start:

- **Measure the per-call token-length distribution first.** If p95 ≤ 8192, use sequence 8192, batch 1, grad accum 8–16, rank 16, alpha 16.
- If p95 is 7k–17k, do **not** train at 4096: most samples would truncate mid-tool-call, teaching the exact failure the prompt fights. Use 8k–16k with FSDP2 BF16 LoRA (Option B), or truncate only at turn boundaries (keep system + last observation + completion).
- 10-step smoke test; measure tokens/s, peak VRAM, loss.
- If 8192 OOMs, reduce sequence only after checking the length distribution; otherwise use 12B/E4B or Option B.

This is the simplest path to a real adapter.

6.3 Option B — 31B LoRA BF16 + FSDP2 on 4×A100 40 GB

Why: BF16 weights are ~61.4 GB total (derived: 30.7B params × 2 bytes, S8; S7's 69.9 GB is inference memory including overhead); sharded across four GPUs that is ~15.4 GB/GPU before activations. FSDP2 + activation offloading + packing at 8k–16k is plausible; 32k is a stretch. Axolotl provides a 31B LoRA FSDP2 example at 32k on multi-GPU (S13), but the example's rank 256 and 32k sequence are not a 4×40 GB recommendation.

Start:

- sequence 8192, batch 1, grad accum 4, rank 16, alpha 16.
- `fsdp_version: 2`, `transformer_layer_cls_to_wrap: Gemma4TextDecoderLayer`, `reshard_after_forward: true`, `activation_offloading: true`.
- 10-step smoke test; watch per-GPU memory and NCCL.
- If it fits, try 16384. If OOM, reduce sequence or rank, or use CPU offload (slower).

Risk: Gemma 4 FSDP has reported gradient/FQN issues in other frameworks (not Axolotl-specific; S50); test before a long run. Axolotl's FSDP+QLoRA docs list only llama/mistral/mixtral, so **do not assume Gemma 4 FSDP+QLoRA works** (S14). BF16 LoRA + FSDP2 is the more likely multi-GPU path.

6.4 Option C — 31B QLoRA + FSDP-QLoRA on 4×A100 40 GB

Theoretically attractive: 4-bit weights ~17.5 GB total, plus FSDP. But Axolotl's FSDP+QLoRA docs do not list Gemma 4 as supported (S14), and multi-GPU QLoRA has `use_reentrant`/memory caveats (S15). Treat this as experimental:

- smoke-test 10 steps at 4096 sequence (smoke only; full training must use the measured p95);
- if it works, compare throughput to Option A;
- if it fails, do not spend days debugging; use Option A or B.

6.5 Option D — 12B QLoRA/LoRA

12B is weaker than 31B in public comparisons (S34), but it trains faster and fits comfortably on one 40 GB card. Use it to:

- validate the data pipeline, masks, and adapter export;
- test alpha/rank sweeps cheaply;
- run more seeds/experiments.

Do not assume a 12B adapter will transfer to 31B; it won't. A 12B result is a pipeline test, not a main-track candidate unless you are targeting a different track.

6.6 Option E — E4B LoRA for the small-model bonus

E4B LoRA is ~17 GB (S10), fits one 40 GB card. The small-model bonus is $10k and the best public score was ~0.06 (S47). Train E4B only if you are deliberately hedging; selecting it consumes one of your two final slots (a probe does not). Use the same data pipeline and validation, but expect weaker repair quality.

6.7 Training-time estimates

There is no official throughput number. Use a 10-step smoke test:

```
tokens/s = (global_batch_size × sequence_length × 10) / elapsed_seconds
```

Then:

```
steps_per_epoch ≈ num_samples / global_batch_size
epoch_time ≈ steps_per_epoch / steps_per_second
```

Plan for tens of GPU-hours for a first 31B campaign, not minutes. [ESTIMATE] A 500–2,000 sample run at 8k sequence, 1–3 epochs, is likely in the tens of GPU-hours on 1×A100 40 GB; multi-GPU FSDP reduces wall time but adds complexity. Measure before booking a long run.

---

7. Recipes

7.1 Axolotl 31B QLoRA (single GPU, adapted from S12)

Use this as a starting point, not a guaranteed-fit config. The official example uses 2k sequence on 1×80 GB; we raise sequence to 4k–8k and lower rank/alpha.

```yaml
base_model: google/gemma-4-31B  # verified BF16 ID (S12); QAT-unquantized alternative requires
ID/revision verification
chat_template: gemma4  # official Axolotl name; verify thinking tokens (see note)
eot_tokens:
  - "<turn|>"
datasets:
  - path: /path/to/your/dataset
    type: chat_template
    field_messages: messages
val_set_size: 0.05
output_dir: ./outputs/gemma4-31b-qlora
sequence_len: 8192          # set from measured p95; do not train at 4k on 7k-17k samples
sample_packing: true

load_in_4bit: true
adapter: qlora
lora_r: 16
lora_alpha: 16              # scale 1.0; sweep 8/16; 32 diagnostic only
lora_dropout: 0
lora_target_modules: 'model.language_model.layers.[\d]+.(_checkpoint_wrapped_module.)?
(mlp|self_attn).(up|down|gate|q|k|v|o)_proj'

micro_batch_size: 1
gradient_accumulation_steps: 8
num_epochs: 1
```

```
optimizer: adamw_torch_8bit
lr_scheduler: cosine
learning_rate: 0.00005
warmup_ratio: 0.1
max_grad_norm: 0.1
weight_decay: 0.0
bf16: auto
tf32: true
gradient_checkpointing: true
activation_offloading: true
gemma4_hybrid_attn_impl: true
attn_implementation: flash_attention_2
```

Notes:

- `base_model` must be a trainable checkpoint, not the `-w4a16-ct` inference checkpoint (S25, S45).
- If the QAT-unquantized checkpoint is not recognized, use `google/gemma-4-31B` BF16 or a compatible mirror; record the revision.
- `gemma4` is Axolotl's official template name (S12). `gemma-4-thinking` is the **model's** template variant (S10), not necessarily an Axolotl `chat_template` value. To train with thoughts, enable the trainer's thinking flag or render with the tokenizer's thinking template, then assert that a decoded sample contains `<|think|>`/`<|channel>thought` tokens. If you cannot, train without thinking and disable thinking at inference; do not mix.
- If you need per-call tokenized samples with custom masks, Axolotl's `chat_template` dataset type may not be enough; use TRL/PEFT directly (S16) or a pre-tokenized dataset.
- `attn_implementation: flash_attention_2` may fail on Gemma 4's global head_dim=512; `gemma4_hybrid_attn_impl: true` is the vendor workaround (S11). If it fails, use `sdpa` (slower, more memory).

7.2 Axolotl 31B LoRA FSDP2 (4×A100 40 GB, adapted from S13)

```
base_model: google/gemma-4-31B
chat_template: gemma4  # official Axolotl name; verify thinking tokens (see note)
eot_tokens:
  - "<turn|>"
datasets:
  - path: /path/to/your/dataset
    type: chat_template
    field_messages: messages
val_set_size: 0.05
output_dir: ./outputs/gemma4-31b-lora-fsdp
sequence_len: 8192          # try 16384; 32768 is a stretch
sample_packing: true

adapter: lora
lora_r: 16
lora_alpha: 16
lora_dropout: 0.05
lora_target_modules: 'model.language_model.layers.[\d]+.(_checkpoint_wrapped_module.)?
(mlp|self_attn).(up|down|gate|q|k|v|o)_proj'

micro_batch_size: 1
gradient_accumulation_steps: 4
num_epochs: 1
optimizer: adamw_torch_8bit
lr_scheduler: cosine
learning_rate: 0.00005
```

```
warmup_ratio: 0.1
max_grad_norm: 0.1
weight_decay: 0.0
bf16: auto
tf32: true
gradient_checkpointing: true
activation_offloading: true
gemma4_hybrid_attn_impl: true
attn_implementation: flash_attention_2

fsdp_version: 2
fsdp_config:
  offload_params: false
  auto_wrap_policy: TRANSFORMER_BASED_WRAP
  transformer_layer_cls_to_wrap: Gemma4TextDecoderLayer
  state_dict_type: SHARDED_STATE_DICT
  final_state_dict_type: FULL_STATE_DICT
  reshard_after_forward: true
```

Launch with Axolotl's multi-GPU command (typically `accelerate launch` or `axolotl train config.yml` with `CUDA_VISIBLE_DEVICES=0,1,2,3`). Check the current Axolotl docs for the exact launcher; do not assume.

7.3 Unsloth 31B QLoRA (single GPU)

Unsloth is the fastest path to a single-GPU 31B QLoRA (S10). The following is **pseudo-code**: verify argument names (`tokenizer` vs `processing_class`, `max_seq_length` vs `max_length`) against the installed Unsloth/TRL versions and record those versions in the manifest.

```
from unsloth import FastModel
from trl import SFTTrainer, SFTConfig

model, tokenizer = FastModel.from_pretrained(
    model_name="unsloth/gemma-4-31B-it",  # [VERIFY] exact mirror; or a compatible 31B trainable checkpoint
    max_seq_length=8192,
    load_in_4bit=True,
    full_finetuning=False,
)

model = FastModel.get_peft_model(
    model,
    finetune_vision_layers=False,
    finetune_language_layers=True,
    finetune_attention_modules=True,
    finetune_mlp_modules=True,
    r=16,
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
)

# dataset is the §6.3 output with input_ids/attention_mask/labels
from datasets import load_from_disk
dataset = load_from_disk("/path/to/samples")

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,  # older TRL; newer versions use processing_class
    train_dataset=dataset,
    args=SFTConfig(
        max_length=8192,  # verify the field name in your TRL version
        per_device_train_batch_size=1,
```

```
            gradient_accumulation_steps=8,
            warmup_steps=5,
            num_train_epochs=1,
            learning_rate=5e-5,
            logging_steps=1,
            optim="adamw_8bit",
            weight_decay=0.001,
            lr_scheduler_type="cosine",
            seed=3407,
            report_to="none",
        ),
    )
    trainer.train()
```

Notes:

- Unsloth's official example uses `r=8`, `alpha=8`; we use 16/16 for more capacity. Start smaller if memory is tight.
- Optimizer names are framework-specific: Unsloth uses `adamw_8bit`; Axolotl uses `adamw_torch_8bit`. Do not copy one into the other without checking.
- If you train with thoughts, use the tokenizer's thinking template (S10) and verify the rendered tokens; `gemma-4-thinking` is a model-template name, not a trainer argument.
- `train_on_responses_only` can mask instruction tokens, but for multi-turn tool calls with custom labels, prefer pre-tokenized `input_ids`/`labels` (S25).
- Unsloth multi-GPU is via Accelerate/DDP/DeepSpeed and is less mature (S10); for 4×A100, Axolotl FSDP2 is more predictable.

7.4 12B / E4B

Do **not** copy the 31B recipe verbatim. Substitute at least these fields:

Field	31B (§7.1/§7.2)	12B / E4B
`base_model`	`google/gemma-4-31B` (verified BF16 ID, S12)	official 12B/E4B checkpoint `[VERIFY exact repo ID/revision]`
`chat_template`	`gemma4` + thinking check	`gemma4`; S10 uses the non-thinking template for small models
`lora_target_modules`	31B text-backbone regex	re-derive from `model.named_modules()` for the checkpoint
`transformer_layer_cls_to_wrap`	`Gemma4TextDecoderLayer`	verify the decoder-layer class per checkpoint
`sequence_len`	8192	8192, set from measured p95

12B: QLoRA/LoRA on one 40 GB card at 8k; rank 16. E4B: LoRA/QLoRA on one 40 GB card; freeze vision/audio; rank 16. 12B/E4B adapters cannot be used with a 31B base; the single-model rule and module shapes prevent it. They are separate submissions/tracks.

---

8. Adapter export

After training:

1. Save the PEFT adapter:

```
model.save_pretrained("adapters/main_lora")
tokenizer.save_pretrained("adapters/main_lora")
```

2. Keep only `adapter_config.json` and `adapter_model.safetensors` in the submission. Remove optimizer states, tokenizer files if not needed, and any training checkpoints.
3. Inspect keys:

```
from safetensors.torch import load_file
sd = load_file("adapters/main_lora/adapter_model.safetensors")
print(len(sd), sorted(sd)[:5])
assert all("lora" in k for k in sd), [k for k in sd if "lora" not in k][:10]
```

4. Check rank/size:

```
du -sh adapters/main_lora
python - <<'PY'
import json
c=json.load(open("adapters/main_lora/adapter_config.json"))
print({k:c.get(k) for k in
["r","lora_alpha","target_modules","base_model_name_or_path","peft_type","task_type"]})
PY
```

5. Ensure `base_model_name_or_path` is the training base (for reproducibility), and `r`/`lora_alpha` are what you intend. The scorer routes by module names, but an accurate config helps debugging.
6. Copy to `adapters/main_lora/` in the submission; reference `adapter: main_lora` on the agent(s).
7. Validate the submission directory with `adk_submission.validate_directory` and the zip size/extension checks (S6:593-611).

If the adapter contains vision/audio tower tensors, strip them for a text-only submission; they are unused and waste space. Check that the remaining keys match vLLM's supported target modules (S22).

---

9. Serving validation on the exact W4A16 base

This is the step most teams skip. Do it before spending a submission slot.

9.1 Start vLLM like the scorer

Use the **same wheelhouse vLLM build** as the scorer/harness, not an arbitrary local install. Record `vllm.__version__` and the wheel hash in the manifest; the Gemma 4 LoRA fix may be present or absent

in either build (S23). The starter notebook's config (S6:272-290):

- model: `google/gemma-4-31B-it-qat-w4a16-ct` (local HF path; declare the competition alias `gemma-4-31b-it-qat-w4a16-ct` in YAML)
- `max_model_len=32768`
- `gpu_memory_utilization=0.90`
- `tensor_parallel_size=4`
- `tool_call_parser='gemma4'`, `reasoning_parser='gemma4'`
- `enable_lora=True`; `max_loras` and `max_lora_rank` are derived from the adapter map (S5:153). For one rank-16 adapter the derived values are 1 and 16; do not hardcode them for other ranks.
- `VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=1`
- `startup_timeout=1200`

If you use the competition's `VllmServer`, it builds the command; don't hand-roll a different stack and assume parity.

9.2 Check startup and KV cache

Look for:

- server healthy within 1200 s;
- `GPU KV cache size` (tokens);
- no sampler warm-up OOM;
- adapter listed in `/v1/models`.

With a rank-16 adapter, community measurements show KV cache around 27,712 tokens at util 0.80 (S27). At 0.90 it may be higher [INFERENCE], but still below the no-adapter baseline; measure it before relying on it. If your prompts routinely exceed the KV cache, requests will queue and tasks will time out. The harness compacts at 14,336 tokens, so most prompts should fit; still test a 28k-token prompt.

9.3 A/B the adapter

At temperature 0, same prompt:

1. Base model, 3 prompts; save outputs/hashes.
2. Adapter model, same prompts; save outputs/hashes.
3. They must differ. If identical, the adapter is ignored (S23).
4. Restart the server and repeat; outputs should be stable (S24). If they vary at temperature 0 across restarts, the W4A16 LoRA path may be unstable; try lower rank/partial coverage.

9.4 Alpha sweep

Pin **rank 16** for the alpha sweep. Edit `adapter_config.json`:

- `lora_alpha = 8` (scale 0.5)
- `lora_alpha = 16` (scale 1.0)
- `lora_alpha = 32` (scale 2.0) — **diagnostic only**, because scale 2.0 is the documented degenerate point at rank 16 (S25)

If you use rank 8, sweep alpha 4/8, not 8/16/32. Restart and run a small task set (5–10 tasks). Measure:

- loops / repeated calls;
- time to first edit;
- patch rate;
- resolved rate;
- thought length.

Public evidence: scale 2.0 degenerated; 1.0 normal but slower; 0.5 safest/fastest but weaker correction (S25). Pick the best trade-off for your data, not the theoretically “full” correction.

9.5 Local paired evaluation

Run base vs adapter on 20–40 holdout tasks:

- same tasks, same seeds, alternate order;
- record resolved, patch rate, p50/p95 time, calls, malformed calls, repeated-call streaks;
- use a sign test on discordant pairs; one or two task differences are noise (S39);
- require **≥8 discordant pairs and ≥75% wins** (or two seeds) and no p95/patch-rate regression; a 2-task net win at 20 tasks is not enough (see `06-experiments-and-tracking.md` §3.4).

Then submit the adapter as one final candidate. Keep the prompt-only bundle as the other. Do not select the adapter until it has a hidden score and a local win.

---

10. Advanced methods

10.1 Step Rejection Fine-Tuning (SRFT)

Use unresolved trajectories but mask harmful steps with a critic; keep the steps as context (S18). On SWE-bench Verified, SRFT 32.2% vs RFT 30.9% (Qwen2.5-Coder-32B + SWE-agent). This is a modest gain, requires a critic, and the critic can be wrong. Use only after a clean SFT baseline; it is not a first experiment.

10.2 SWE-Prime-style data selection

Score trajectories and segments for process quality, result quality, representativeness, contribution, learnability, and risk; keep all segments in context but only selected segments contribute to loss (S19). A 10% selected subset outperformed the full resolved set in the paper. This is a data-quality lever, not a new model. It is compatible with LoRA SFT.

10.3 Distillation from stronger models

The rules allow external models unless prohibited, but external data must be public/equally accessible or meet the Reasonableness Standard (S3). Distillation can work, but:

- check the teacher’s license and the competition rules;

- publish the generated dataset and generation code if you use it;
- clean leakage (teacher may read fixed copies outside `/workspace`);
- do not assume a stronger teacher's trajectories are high-quality; filter them (S19).

10.4 RL / GRPO / DPO

RL with test-based rewards is attractive but expensive and complex:

- you need a reliable reward (tests pass) and a sandbox for rollouts;
- Gemma 4 + FSDP/GRPO has reported instability in other frameworks (S50);
- the competition's 12h inference budget does not apply to training, but your training budget does;
- no public competition evidence shows RL beating a strong SFT/prompt baseline.

Treat RL as a research bet, not a first path. If you try it, start with a small E4B/12B model and a narrow reward, and keep the SFT/prompt baseline as your submission.

---

11. Blockers and mitigations

Blocker	Evidence	Detection	Mitigation
PEFT cannot attach to CT checkpoint	S25, S45	Loader error / no trainable params	Train on unquantized QAT/BF16; serve adapter on CT
QLoRA quantizer mismatch	S45	Adapter loads but behavior differs	Validate on CT; prefer BF16 LoRA if memory allows
W4A16 rank instability	S24	Weak/non-reproducible outputs at temp 0	Rank 8–16; partial coverage; test
Adapter silently ignored	S23	Base/adapter outputs identical	A/B at temp 0; pin vLLM/base revision
Alpha scale degenerate	S25	Loops, long thoughts, no edit	Alpha = rank; sweep 8/16; 32 diagnostic
Thinking mismatch	S25	Huge thoughts/timeouts	Train and infer with same thinking mode
Startup OOM	S26	Sampler warm-up OOM	`VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=1`; one adapter; low rank
KV cache drop	S26, S27	Long prompts queue	Test 28k prompt; one adapter; compaction; lower rank
FSDP+QLoRA Gemma 4 unsupported	S14	Trainer errors/OOM	Use single-GPU QLoRA or BF16 LoRA FSDP2; smoke test

Blocker	Evidence	Detection	Mitigation
Gradient spikes	S11	Grad norm 100–1000×	LR 5e-5; max_grad_norm 0.1
Data too small/weak	S25	Adapter loses to base	More/better trajectories; thinking consistency; quality filtering
Training data rules	S3	Winner disclosure	Publish data/code; use public data; keep manifests
Compute/storage	[ESTIMATE]	OOM/disk full	Preflight; 250–400 GB; measure tokens/s
Hidden score noise	S34, S35, S39	One run moves ±2 tasks	Paired local; two final submissions; don't overfit

12. Stop criteria

Stop the post-training line and keep the prompt-only baseline if:

- by **Nov 13** the adapter has not passed A/B on the exact base and a paired holdout with ≥8 discordant pairs/≥75% wins;
- by **Nov 20** the adapter has no hidden score;
- the adapter improves resolve rate but increases p95/mean time enough to threaten the 12h limit;
- the adapter causes loops/timeouts that alpha/rank sweeps cannot fix;
- the adapter cannot be validated on the exact W4A16 base (ignored/unstable/startup OOM);
- you have fewer than ~300 high-quality resolved trajectories and no clear path to more.

Calendar gates: **Nov 6** calibration ≥90% or fallback sandbox; **Nov 13** adapter validation gate; **Nov 20** hidden score; **Nov 24** no new experiments; **Nov 28** freeze. If the best hidden score is <0.12 by Nov 25, stop optimizing and spend the remaining slots on diverse prompt/budget variants.

A negative result is still useful for the paper track. The public 1,500-trajectory experiment is a valuable case study precisely because it documented what didn’t work (S25).

13. Post-training checklist

- ☐ Prompt-only baseline scored and saved.
- ☐ Calibrated local evaluator; gold patches pass.
- ☐ Training base is unquantized QAT or BF16, not CT.
- ☐ Base revision pinned; adapter config records it.
- ☐ Trajectories generated with the same prompt/tools/thinking as inference.
- ☐ Resolved-only filtering; shortest/quality selection; leakage checks.
- ☐ Assistant-only loss verified on a decoded sample.
- ☐ Sequence length and packing verified; no mid-tool-call truncation.
- ☐ 10-step smoke test: loss finite, VRAM stable, tokens/s measured.
- ☐ Full run: loss curve sane, no gradient explosion; checkpoint saved.
- ☐ Adapter keys only LoRA; rank/size/config checked.

- ☐ vLLM startup with `VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=1`; KV cache measured.
- ☐ A/B at temp 0 shows output difference; stable across restart.
- ☐ Alpha sweep 8/16 (rank 16); 32 diagnostic; rank 8/16 compared if time.
- ☐ Paired local eval ≥8 discordant pairs/≥75% wins (or two seeds); p95/patch rate not worse; hidden score before final selection.
- ☐ Adapter submission as one final candidate; prompt-only baseline kept as the other.
- ☐ Data/code/license/manifest ready for winner disclosure.